

上板成功

MPU-6050 IIO 驱动 · 真实硬件里程碑

从零手写的驱动,在 STM32MP157 + 真 MPU-6050 上读出真实数据

配套仓库 github.com/fyy0619-cell/xiangmu · 2026 年 10 月

一、成果:在真板子上跑通,读出真实数据

自己从零手写的 MPU-6050 IIO 驱动,经交叉编译部署到 STM32MP157(正点原子 ATK),自改设备树挂到 I2C4,在真实硬件上成功探测芯片并读出随姿态变化的真实加速度/角速度/温度。不再是模拟或编译通过,而是"做出来了"。

关键证据(板子串口终端实录)

```
[root@ATK-stm32mp1]:/$ insmod /mpu6050_iio.ko
mpu6050_iio: loading out-of-tree module taints kernel.
mpu6050_iio 0-0068: MPU-6050 detected!           # 真芯片 WHO_AM_I 读对
mpu6050_iio 0-0068: registered as an IIO device

$ cat in_accel_x_raw in_accel_y_raw in_accel_z_raw
992  16768  -1216          # Y 轴朝上:16768 x 0.000598550 ≈ 10.0 m/s² ≈ 1g(重力!)
$ cat in_accel_x_raw in_accel_y_raw in_accel_z_raw  (翻转板子后)
-8928  7324   9720        # 数值完全变化 = 真实加速度随姿态改变

$ cat in_accel_scale
0.000598550
```

物理验证:静止时某轴 $\approx 1g$ (raw ≈ 16384);一翻转数值就变。这就是"真实世界数据",是模拟永远给不了的。

完整 IMU 都可读(驱动支持 accel+gyro+temp)

```
in_accel_{x,y,z}_raw + in_accel_scale      # 加速度 → m/s²
in_anglvel_{x,y,z}_raw + in_anglvel_scale  # 角速度 → rad/s(旋转时变)
in_temp_raw + in_temp_scale + in_temp_offset # 温度 → (raw+12420)/340 °C
```

二、上板做了什么(可复现)

步骤	做法
1. 交叉编译	<code>make KERNELDIR=.../linux-5.4.31</code> (内核 Makefile 已设 ARCH=arm、CROSS_COMPILE=arm-none-linux-gnueabi-) → 得 ARM <code>.ko</code> (file 验证含 ARM)
2. 避免冲突	驱动 <code>of_match</code> 改独有 <code>compatible = "alientek,mpu6050"</code> ,避开主线 <code>inv_mpu6050</code>
3. 设备树	在 <code>stm32mp157d-atk.dts</code> 末尾加 <code>&i2c4 override:打开 I2C4(PZ4/PZ5)+ 挂 mpu6050@68</code> 节点 → <code>make stm32mp157d-atk.dtb</code> → 拷到 tftpboot
4. 接线	MPU6050 SCL→JP1-25(I2C4_SCL/PZ4),SDA→JP1-26(I2C4_SDA/PZ5),VCC→3V3,GND→GND
5. 部署	ARM <code>.ko</code> 拷到 NFS 根文件系统(<code>.../nfs/rootfs/</code>)
6. 加载验证	板子重启(载新 dtb)→ <code>insmod</code> → <code>dmesg</code> 见 detected → <code>cat</code> 读真实数据

设备树节点:

```
&i2c4 {
    pinctrl-names = "default", "sleep";
    pinctrl-0 = <&i2c4_pins_a>;          /* PZ4=SCL, PZ5=SDA, AF6 */
    pinctrl-1 = <&i2c4_pins_sleep_a>;
    clock-frequency = <100000>;
    status = "okay";                    /* 打开 I2C4(默认 disabled) */
    mpu6050@68 {
        compatible = "alientek,mpu6050"; /* 与驱动 of_match 对上 */
        reg = <0x68>;
    };
};
```

三、概念收口:I2C 与 IIO 是什么关系?板上有 IIO 硬件吗?

设备挂在 I2C 总线,却叫 IIO 驱动——因为 I2C 和 IIO 是两个不同维度,一个驱动同时兼两职(图 1)。

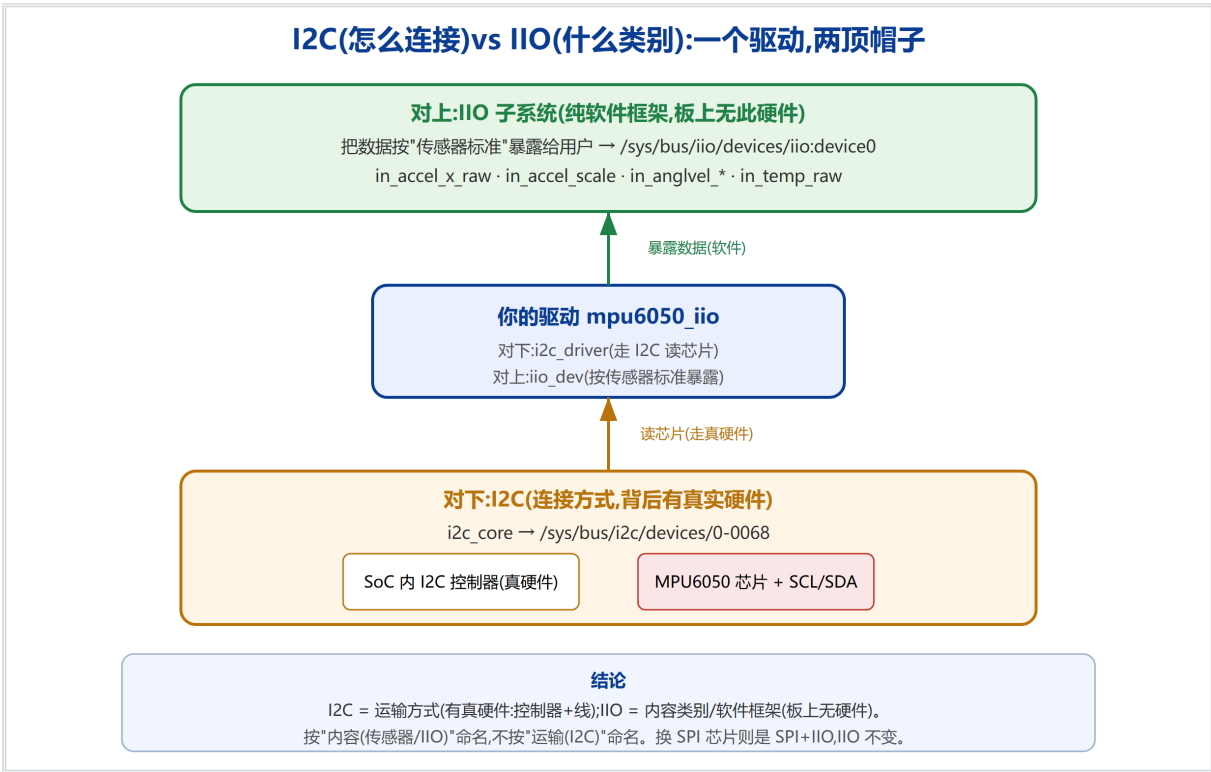


图 1 I2C(怎么连接)vs IIO(什么类别):一个驱动两顶帽子

	I2C	IIO
回答什么	怎么和芯片通信(连接方式)	这是哪类设备、怎么把数据给用户(软件框架)
在驱动里	i2c_driver + regmap 读芯片(对下)	iio_dev + 通道 + read_raw,暴露 /sys/bus/iio(对上)
是硬件吗	是(SoC 内 I2C 控制器 + SCL/SDA 线)	不是 ,纯软件框架(drivers/iio/*.c)

- **为什么叫 IIO 驱动:**按"内容/类别(传感器→IIO)"命名,I2C 只是运输方式。类比:顺丰(I2C)寄来的生鲜(IIO),按内容叫生鲜。
- **同一设备两个视图:** /sys/bus/i2c/devices/0-0068 (连接)与 /sys/bus/iio/devices/iio:device0 (传感器身份)。
- **板上有 IIO 硬件吗?没有。**板上真实硬件只有 MPU6050 + I2C 控制器 + SCL/SDA;IIO 是内核里组织传感器驱动、给用户暴露数据的软件框架, /sys/bus/iio/ 是软件虚拟目录。
- 换成 SPI 接口的芯片就是"SPI + IIO",**IIO 不变、只换总线**——这也是生产级驱动拆 core(IIO)+ i2c(总线)的原因。

四、这意味着什么

整条链路全部打通并经真实硬件验证:**从零手写驱动** → **理解每一层** → **交叉编译 ARM** → **改设备树** → **部署上板** → **读出真实传感器数据**。面试可据实陈述:

"我从零写了一个 MPU-6050 的 Linux IIO 驱动,交叉编译部署到 STM32MP157,自己改设备树把它挂到 I2C4,在真板子上读出了随姿态变化的真实加速度/角速度/温度。"

后续可延伸:DRDY 中断 + 触发缓冲(生产级数据流)、采样率配置、对照主线 inv_mpu6050。

学习版代码 `examples/learning-driver/` (已在真 STM32MP157 验证);完整从零教程见 `docs/learning/` 。仓库:github.com/fyy0619-cell/xiangmu